



Practice with Lists  
and while Loops

# Announcements

## Re: Quiz 01

- Great job!! Median was ~85%
- Graded quizzes will be available on Gradescope soon! Once they're released:
  - *Please review what you missed ASAP*; we will build on the topics covered in Quiz 01 throughout the course, and these foundational concepts are vital!
  - Don't understand a particular question/part of a memory diagram? Please visit us in Office Hours or Tutoring! Full list of hours on the site's [support page](#)
  - Please submit a regrade request *if you believe your quiz was not graded correctly according to the rubric*

***EX03: Wordle due today at 11:59pm***

# Warm-up

```
1  def gen(stop: int) -> list[int]:
2      """Generate a list from 0 to stop, not inclusive."""
3      i: int = 0
4      acc: list[int] = []
5      while i < stop:
6          acc.append(i)
7          i = i + 1
8      return acc
9
10
11 print(gen(3))
```

# Coding an RSVP tracker!

In this in-class exercise, you will manage a global list of friends who have RSVPed “yes” to a party.

First, add this global variable to your .py file:

```
rsvp_list: list[str] = ["Enrico", "Yun"]
```

# add\_rsvp function

**Goal:** Record that a friend has RSVPed to the party.

## Instructions:

Write a function:

```
def add_rsvp(friend: str) -> None:
```

This function should:

1. Take a friend's name as a parameter.
2. Add the friend's name to the global `rsvp_list`.
3. Mutate the list using a list method.

# has\_rsvped

**Goal:** Check whether a specific friend has already RSVPed.

## Instructions

Write a function:

```
def has_rsvped(friend: str) -> bool:
```

This function should:

1. Use a while loop to search through `rsvp_list`.
2. Compare each name in the list to the given `friend`.
3. Return `True` as soon as a match is found.  
Return `False` if the loop finishes without finding the friend.
4. **Do not mutate** the list.

# not\_yet\_rsvped

**Goal:** Determine which invited friends have *not* RSVPed yet.

You will be given a list of all invited friends:

invited: list[str]

## Instructions

Write a function:

```
def not_yet_rsvped(invited: list[str]) -> list[str]:
```

This function should:

1. Create a new empty list to store missing RSVPs.
2. Use a `while` loop to iterate through the `invited` list.  
For each invited friend:
  - Use your `has_rsvped` function to check if they are already in `rsvp_list`.
  - If they have **not** RSVPed, add their name to the new list.
3. Return the new list of friends who still need to RSVP.
4. Do **not** mutate either `invited` or `rsvp_list`.

# Diagramming a Nested List

```
1  def sum2d(xs: list[list[int]]) -> int:
2      """Calculate the sum of a 2-dimensional list of lists."""
3      total: int = 0
4      row_i: int = 0
5      while row_i < len(xs):
6          col_i: int = 0
7          while col_i < len(xs[row_i]):
8              total += xs[row_i][col_i]
9              col_i += 1
10         row_i += 1
11     return total
12
13
14 values: list[list[int]] = [[1, 2, 3], [3, 4, 5]]
15 print(sum2d(values))
```